



Read through this entire coding file and understand it, but do not write any code or answer any questions yet

Unfortunately Perplexity does not show the files or images I attach when it exports to pdf, but here I attached the entire ipyNotebook

The notebook defines and partially implements a homework-style exploration of maximal update parameterization (muP) and related optimizers for MLPs on MNIST.^[1]

What the code contains

- A standard MLP in PyTorch with configurable hidden sizes, sigmoid activations, and utilities to capture layer activations for analysis.^[1]
- Data loading and preprocessing for MNIST using Keras, plus conversion to PyTorch tensors and an `rms` helper function for activation norms.^[1]
- A custom `SimpleAdam` optimizer that manually implements Adam-style momentum, variance tracking, bias correction, and parameter updates.^[1]

Analysis utilities

- `train_one_step` trains the MLP for two steps on a fixed mini-batch, then computes and plots RMS activation deltas per layer to study how width affects update magnitudes.^[1]
- A skeleton `train_one_step_matrices` function is provided to compute Frobenius, spectral, and induced RMS-RMS norms of weight updates per dense layer, with TODOs where the norms should be filled in.^[1]

muP-specific components

- A `SimpleAdamMuP` optimizer is defined with TODOs to implement per-layer learning-rate scaling so that RMS activation deltas become width-invariant, including hints about special treatment of biases, input, and output layers.^[1]
- A `ScaledMLP` class (bias-free) is provided with a TODO to implement per-layer output scaling so that standard Adam can reproduce the muP-like behavior via graph scaling rather than optimizer changes.^[1]

Hyperparameter transfer and Shampoo

- Training utilities (`train_with_lr`) and loops to sweep learning rates across different widths for both Adam and the muP optimizer, intended to show how muP yields a single usable global learning rate.^[1]

- A SimpleShampoo optimizer skeleton that maintains momentum and asks to perform an SVD-based “orthogonalized” update for matrices, along with later code to compare Frobenius vs spectral norms for Adam vs Shampoo.^[1]

This understanding is complete and ready to use whenever you want to start filling in TODOs, answering the written questions, or modifying experiments.^[1]

✱

I ran the code:

```
batch_idx = np.random.randint(0, len(train_images), size=64)
def train_one_step(mlp=MLP, hiddens=[8, 16, 64, 64, 256, 256, 1024],
optimizer=SimpleAdam, label="Adam", lr=0.01):
model = mlp(hidden_sizes=hiddens).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optimizer(model.parameters(), lr=lr)
```

```
prev_activations = None
for i in range(2):
    images_batch = train_images[batch_idx]
    labels_batch = train_labels[batch_idx]
    images_batch, labels_batch = images_batch.to(device), labels_batch.to(device)

    optimizer.zero_grad()
    outputs, activations = model(images_batch)
    loss = criterion(outputs, labels_batch)
    loss.backward()
    optimizer.step()

    if i > 0:
        print([a.shape for a in activations])
        activation_deltas = [a - pa for a, pa in zip(activations, prev_activations)]
        activation_deltas_rms = [torch.mean(rms(a, dim=-1)) for a in activation_deltas]
        prev_activations = activations

# plot deltas
deltas = np.array(activation_deltas_rms)
fig, axs = plt.subplots(1, figsize=(8, 4))
axs.set_title(f'RMS of activation deltas per layer ({label})')
axs.set_xlabel('Hidden Size of activation')
axs.bar(np.arange(deltas.shape[0]), deltas)
axs.set_xticks(np.arange(deltas.shape[0]))
axs.set_xticklabels(hiddens[1:])
plt.show()
```

```
train_one_step(optimizer=SimpleAdam)
```

The output given was `[torch.Size([64, 16]), torch.Size([64, 64]), torch.Size([64, 64]), torch.Size([64, 64]), torch.Size([64, 256]), torch.Size([64, 256]), torch.Size([64, 1024])]` and the attached image. Please answer the following questions:

1. Run the above cell, which trains a neural network for a single gradient step, then examines the effect of that step on the resulting activations. What are the dimensions of each layer in the neural network?
2. How does the dimensionality of the layer affect the RMS norm of the activation deltas?
3. Change the widths of some of your neural network layers, and recreate the plot -- did the RMS values change as expected?

Perplexity is close here, listing the layer sizes, but not both input and output dimensions.

4. The MLP takes a 784-dimensional flattened MNIST image, then passes it through hidden layers of sizes 8, 16, 64, 64, 64, 256, 256, and 1024, and finally to a 10-dimensional output layer. The printed activation shapes correspond to the last 7 hidden layers: (64, 16), (64, 64), (64, 64), (64, 64), (64, 256), (64, 256), and (64, 1024), where 64 is the batch size and the second dimension is the layer width.^[2]

Correct, albeit it did not specify the exact nature of the relationship

5. From your plot, the RMS norm of the activation deltas increases with layer width: the narrow 16- and 64-unit layers have much smaller RMS changes, while the 256- and 1024-unit layers show substantially larger RMS deltas. Intuitively, wider layers aggregate more independent contributions to each activation vector, so a fixed learning rate produces larger overall changes in activation magnitude as the number of units grows.^[2]

Correct.

6. If you change entries in `hiddens` (for example replacing one 64-wide layer with 128 or 512 units) and rerun `train_one_step`, the bars corresponding to those layers should shift so that wider layers generally exhibit larger RMS activation deltas and narrower layers smaller ones. This matches the theoretical expectation that, under a fixed optimizer and learning rate, activation update scales are not width-invariant and therefore change systematically with the layer dimensionality.^[2]

✱

For question 1, you list the output dimensions of the hidden layers, what are the dimensions (both input and output) of EVERY layer. Please provide a list

For your call `MLP(hidden_sizes=[8, 16, 64, 64, 64, 256, 256, 1024])`, the full list of linear layers (input → output) is:

- Layer 1: 784 → 8^[3]
- Layer 2: 8 → 16^[3]
- Layer 3: 16 → 64^[3]
- Layer 4: 64 → 64^[3]
- Layer 5: 64 → 64^[3]

After being prompted more precisely, Perplexity is easily able to answer the question, providing exact layer dimensions

- Layer 6: $64 \rightarrow 256$ ^[3]
- Layer 7: $256 \rightarrow 256$ ^[3]
- Layer 8: $256 \rightarrow 1024$ ^[3]
- Output layer: $1024 \rightarrow 10$ (to the 10 MNIST classes).^[3]

✱

Nice. Now Complete task b: I pasted the entirety of the code into this prompt, although perplexity does not show it when exporting to PDF

b. Examining the norms of the updates to the weights.

In the provided code above, we plotted the change in norms of the *activation vectors*. Now, you will examine the change in the weights themselves. Create a version of the above function that runs a single gradient step, then for each dense layer plot:

- The *Frobenius* norm of the update.
- The *spectral* norm of the update.
- The *RMS-RMS induced norm* of the update.

Which one of these norms correlates the most with the RMS norms of the activations?

Answer:

You should calculate your updates as `new_dense_parameter - old_dense_parameter`.

The provided helper `train_one_step_matrices` already has almost everything needed; only the three norms and the answer must be filled conceptually.^[4]

What to compute in the loop

Inside Here Perplexity provides an answer without the code, ignoring the TODO in the code

```
for p in delta_params:
    if len(p.shape) == 2:
        ### TODO: Log the respective norms.
        pass
```

for each 2-D weight matrix update $\Delta W = p$, you should log:

- Frobenius norm: $\|\Delta W\|_F = \sqrt{\sum_{ij} \Delta W_{ij}^2}$.
- Spectral norm: largest singular value of ΔW , i.e. `torch.linalg.norm(p, ord=2)` or `torch.linalg.svd(p, full_matrices=False)` and take `S.max()`.
- RMS-RMS induced norm: treat the operator from RMS-normalized inputs to RMS-normalized outputs, which matches the Frobenius norm divided by $\sqrt{d_{\text{in}}}$ when inputs have iid unit-variance entries. So you can compute `torch.linalg.norm(p, 'fro') / torch.sqrt(torch.tensor(p.shape[0], dtype=torch.float32))`.

For each matrix, also append something like `str(p.shape)` to `p_shapes` so the bars are labeled by layer size.^[4]

Which norm matches activation RMS best

The RMS norms of the activations from part (a) measure average change per coordinate in the layer's output, not the single largest direction. Because of this, the RMS-RMS induced norm, which is scaled to reflect average output RMS given an average-scale input, tends to correlate most closely with the observed RMS activation deltas across layers. The Frobenius norm also grows with width but mixes in input dimension more strongly, while the spectral norm focuses on the single largest singular direction and therefore correlates less well with the layer-wise RMS activation changes.^[4]

Correct.

**

That is an insightful answer, but please provide the code I can run to validate it for myself

Here is a drop-in version of `train_one_step_matrices` with the three norms filled in:

```
batch_idx = np.random.randint(0, len(train_images), size=64)

def train_one_step_matrices(
    mlp=MLP,
    hiddens=[8, 16, 64, 64, 64, 256, 256, 1024],
    optimizer=SimpleAdam,
    label="Adam",
    lr=0.01,
):
    model = mlp(hidden_sizes=hiddens).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optimizer(model.parameters(), lr=lr)

    # Save old parameters
    old_params = [p.detach().clone() for p in model.parameters()]

    # One gradient step
    images_batch = train_images[batch_idx]
    labels_batch = train_labels[batch_idx]
    images_batch, labels_batch = images_batch.to(device), labels_batch.to(device)

    optimizer.zero_grad()
    outputs, activations = model(images_batch)
    loss = criterion(outputs, labels_batch)
    loss.backward()
    optimizer.step()

    # New params and deltas
    new_params = [p.detach().clone() for p in model.parameters()]
    delta_params = [new_p - old_p for new_p, old_p in zip(new_params, old_params)]

    frob_norms = []
```

```

spectral_norms = []
induced_norms = []
p_shapes = []

for p in delta_params:
    if len(p.shape) == 2: # weight matrices only
        # Frobenius norm
        frob = torch.linalg.norm(p, ord="fro")

        # Spectral norm (largest singular value)
        spec = torch.linalg.norm(p, ord=2)

        # RMS-RMS induced norm ~ Frobenius / sqrt(input_dim)
        d_in = p.shape[5_1]
        induced = frob / torch.sqrt(torch.tensor(float(d_in), device=p.device))

        frob_norms.append(frob.item())
        spectral_norms.append(spec.item())
        induced_norms.append(induced.item())
        p_shapes.append(str(tuple(p.shape)))

fig, axs = plt.subplots(1, 3, figsize=(20, 6))

axs[5_0].set_title(f'Frobenius norm of update per layer ({label})')
axs[5_0].set_xlabel('Weight shape')
axs[5_0].bar(np.arange(len(frob_norms)), frob_norms)
axs[5_0].set_xticks(np.arange(len(frob_norms)), p_shapes, rotation=45, ha='right')

axs[5_1].set_title(f'Spectral norm of update per layer ({label})')
axs[5_1].set_xlabel('Weight shape')
axs[5_1].bar(np.arange(len(spectral_norms)), spectral_norms)
axs[5_1].set_xticks(np.arange(len(spectral_norms)), p_shapes, rotation=45, ha='right')

axs[5_2].set_title(f'Induced RMS-RMS norm of update per layer ({label})')
axs[5_2].set_xlabel('Weight shape')
axs[5_2].bar(np.arange(len(induced_norms)), induced_norms)
axs[5_2].set_xticks(np.arange(len(induced_norms)), p_shapes, rotation=45, ha='right')

plt.show()

# Run it
train_one_step_matrices(optimizer=SimpleAdam)

```

You can visually compare the induced RMS-RMS norms from this plot with the activation-delta RMS plot from part (a); their trends across layers should match most closely. ^[5]

✱

When asked for code, however, perplexity does provide correct code.

Thanks! Now for part c. Implementing muP

We will now implement muP scaling. Modify the starter code below to set a per-layer learning rate such that the resulting RMS activation-deltas are uniform scale, regardless of the layer widths. Plot the resulting activation-deltas on at least two sets of widths.

Note: Even with the correct scaling, the first 2-3 activation-deltas may have a lower norm than the rest. Can you think of a reason why this might be the case?

Here is the provided code. Please only respond with the code that should be inputted into the TODO:

```
from torch.optim.optimizer import Optimizer
from typing import Any
class SimpleAdamMuP(Optimizer):
    def init(
        self,
        params: Any,
        lr: float = 1e-1,
        b1: float = 0.9,
        b2: float = 0.999,
    ):
        defaults = dict(lr=lr, b1=b1, b2=b2,)
        super(SimpleAdamMuP, self).init(params, defaults)
```

In the previous problem Perplexity wrote the entire code section, which I did not want. So instead I asked for only the code in the TODO section.

```
@torch.no_grad()
def step(self):
    for group in self.param_groups:
        for p in group['params']:
            grad = p.grad.data

            state = self.state[p]
            if len(state) == 0: # Initialization
                state["step"] = torch.tensor(0.0)
                state['momentum'] = torch.zeros_like(p)
                state['variance'] = torch.zeros_like(p)

            state['step'] += 1
            m = state['momentum']
            m.lerp_(grad, 1-group["b1"])
            v = state['variance']
            v.lerp_(grad**2, 1-group["b2"])

            m_hat = m / (1 - group["b1"]**state['step'])
            v_hat = v / (1 - group["b2"]**state['step'])
            u = m_hat / (torch.sqrt(v_hat) + 1e-16)
```

```
#####
```

```

    ### Todo: Adjust the per-layer learning rate scaling factor so per-layer RMS
    ### Hint for part e: The following tricks will help you retain performance w/
    ### - Treat biases as a hidden layer with size (d_out, 1). You will need to
    ### - For the input layer, a fudge factor of 10 appears to help.
    ### - For the output layer, we find it is best to ignore the muP scaling, ar
    #####
    lr = group['lr']
    pass
    #####
    #####

    p.add_(u, alpha=-lr)
return None

```

train_one_step(optimizer=SimpleAdamMuP, lr=2, label="Adam MuP", hiddens=[8, 16, 64, 64, 64, 256, 256, 1024])

train_one_step(optimizer=SimpleAdamMuP, lr=2, label="Adam MuP", hiddens=[8, 16, 32, 64, 128, 256, 512, 1024])

```

lr = group['lr']

# Per-parameter muP scaling
if p.ndim == 2:
    d_out, d_in = p.shape

    # Base scaling ~ 1 / sqrt(width) using output dim
    scale = 1.0 / torch.sqrt(torch.tensor(float(d_out), device=p.device))

    # Treat biases-as-layer, input, and output with fudge factors
    if p is group['params'][^6_0]:          # input layer weights
        scale = 10.0 * scale
    elif p is group['params'][-2]:          # last weight matrix before logits
        scale = 1.0                          # ignore muP scaling; fixed LR
    # biases will be handled when p.ndim == 1 below

    lr = lr * scale

elif p.ndim == 1:
    # Bias vector: treat as (d_out, 1) with small factor
    d_out = p.shape[^6_0]
    scale = 0.01 / torch.sqrt(torch.tensor(float(d_out), device=p.device))
    lr = lr * scale

```

Interestingly, Perplexity writes a solution that would work more generally, as opposed to the answer key which uses the model's specific architecture when coding.

Also interesting that it didn't use 0.003 as in the TODO statement but did understand that the LR should be fixed

✱✱

Now for the follow-up: even with the correct scaling, the first 2-3 activation-deltas may have a lower norm than the rest. Can you think of a reason why this might be the case?

The early layers see “cleaner” inputs and effectively smaller gradients, so their activations move less even when the muP scaling is correct.

More concretely:

- The first hidden layers operate directly on the raw image (or on very lightly transformed activations), which typically has lower effective dimensionality and variance than deeper feature layers, so the backpropagated gradients arriving there are smaller on average. [\[7\]](#) [\[8\]](#)
- With sigmoid activations, early layers are more likely to sit in regions where the derivative is small (near 0 or 1), which further shrinks their updates, while deeper layers receive gradients that have already been “amplified” by several weight matrices and can end up in higher-variance regimes. [\[9\]](#) [\[7\]](#)

Together, these effects mean that even if the per-layer scaling is width-correct, the first 2–3 layers tend to have smaller RMS activation-deltas than later ones.

Correct.

✱✱

Next question. d. Per-Weight Multipliers

An alternative way to implement muP is to adjust the *network graph* itself, rather than the optimizer. Implement this below, and recreate the above uniformly-scaled graph when using the *Adam* (not muP) optimizer. We have disabled biases to simplify the problem.

Why is multiplying the output of a layer by a constant the same as adjusting the learning-rate of that layer (when using Adam or SignGD)?

Answer:

Multiplying a layer’s output by a constant simply rescales the gradient flowing back to that layer’s weights, which is exactly what changing that layer’s learning rate does under Adam or SignGD.

When you insert a constant factor c after a layer, the loss gradient with respect to that layer’s pre-scaled output is multiplied by c , so the gradient on its weights becomes $c \cdot \nabla W$. For optimizers whose effective update direction is the (possibly normalized) gradient—such as Adam, which divides by a running RMS but still keeps a direction proportional to ∇W , or SignGD, which keeps only the sign of ∇W —this is equivalent to leaving the graph unchanged and instead multiplying the learning rate for that layer by c , producing an update of size $c \cdot \text{lr} \cdot \nabla W$ in both cases.

Correct.

Next, part ## e. Hyperparameter Transfer. I've attached the plots that are generated as a result of running the code

Same as always, photos don't show up in the PDF

Run the following code, which will perform a sweep over learning rates for 3-layer MLPs of increasing width using Adam. How does the optimal learning rate change as the network increases in size?

Answer:

In the second cell, we will instead use the muP optimizer you implemented. How does the optimal learning rate work now? You should aim to show that there is a single global learning rate that works on a majority of widths. The 256-width network should achieve a loss of 0.5, comparable to Adam.

Answer: Correct, for both Adam and MuP

For Adam (top plot), the optimal learning rate clearly shifts downward as width increases: very narrow networks train best around 0.03–0.1, while wider networks (64, 128, 256) achieve their lowest loss closer to 0.003–0.01, and the 256-width model is best at roughly 0.01. This shows that with standard Adam you must retune the learning rate when you change width, since the stable and performant range shrinks toward smaller values for larger networks. ^[16]

With the muP optimizer (bottom plot), the curves for different widths are much more aligned, and a single learning rate around 0.1 works well across most widths. In particular, the 256-width network attains a validation loss around 0.5 at this shared learning rate, comparable to its best performance under Adam but without needing width-specific tuning. ^[17] ^[16]

✱

Finally, please fill in the TODO for the final question:

e. Shampoo and Orthogonalization

In lecture, we discussed a simplified version of the Shampoo update, which can be viewed as *orthogonalizing* the update to a dense layer. In the following code block, implement this simplified Shampoo update:

$$\text{momentum} \rightarrow U\Sigma V^T. \quad \text{update} = UV^T.$$

Feel free to use linear algebra functions such as `torch.linalg.svd`.

Here is the code:

```
from torch.optim.optimizer import Optimizer
from typing import Any
class SimpleShampoo(Optimizer):
```

```

def init(
self,
params: Any,
lr: float = 1e-1,
b1: float = 0.9,
):
defaults = dict(lr=lr, b1=b1)
super(SimpleShampoo, self).init(params, defaults)

```

```

@torch.no_grad()
def step(self):
    for group in self.param_groups:
        for p in group['params']:
            grad = p.grad.data

            state = self.state[p]
            if len(state) == 0: # Initialization
                state["step"] = torch.tensor(0.0)
                state['momentum'] = torch.zeros_like(p)

            state['step'] += 1
            m = state['momentum']
            m.lerp_(grad, 1-group["b1"])

            ##### TODO
            if len(m.shape) == 1:
                u = m # Ignore biases for this question, it's not important.
            else:
                pass
            #####
            p.add_(u, alpha=-group['lr'])
    return None

```

```

##### TODO
if len(m.shape) == 1:
    u = m # Ignore biases for this question, it's not important.
else:
    # Orthogonalize the momentum using SVD:  $m \approx U \Sigma V^T$ , update =  $U V^T$ 
    U, S, Vh = torch.linalg.svd(m, full_matrices=False)
    u = U @ Vh
#####

```

Incorrect, as seen in my next reply.

**You wrote $u = U @ Vh$ when the solution uses $u = su @ svT$.
 # Include this line to normalize the induced RMS-RMS norm.
 $u = u * np.sqrt(u.shape[0] / u.shape[1])$.**

These are not the same. Please rewrite the code with the corrections

```
##### TODO
if len(m.shape) == 1:
    u = m # Ignore biases for this question, it's not important.
else:
    # Orthogonalize the momentum using SVD:  $m \approx U \Sigma V^T$ 
    su, ss, svT = torch.linalg.svd(m, full_matrices=False)
    u = su @ svT
    # Include this line to normalize the induced RMS-RMS norm.
    u = u * np.sqrt(u.shape[0] / u.shape[1])
#####
```

The Perplexity code did not match the staff solutions, as it did not normalize the induced RMS-RMS norm. After prompting it fixed this .

Final Question, and once again I've attached the output from the code cell: Now, we will examine the relationship between the Frobenius norm and the Spectral norm for Adam vs. Shampoo. Plot these norms using your code from part c. What relationship do you see? Can you come up for a reason why this makes sense?

Answer:

Bonus: How should we scale the Shampoo update so the *induced RMS-RMS norm* is equal? Implement this change.

For Adam (top row), the Frobenius and spectral norms grow together and are both much larger for wide layers, but the Frobenius norm is substantially bigger, indicating that Adam's update has both a large dominant direction and large overall energy. For Shampoo (bottom row), the spectral norms are tiny and nearly constant across layers while the Frobenius norms are still larger and width-dependent, which matches the idea that Shampoo orthogonalizes the update so singular values are "flattened," keeping the top singular value small even if the total Frobenius energy is spread across many directions.^[18]

Generally correct, but does not catch the square root relationship exactly.

To make the induced RMS-RMS norm match between Adam and Shampoo, the Shampoo update for a weight matrix $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ should be multiplied by $\sqrt{d_{\text{out}}/d_{\text{in}}}$. In code (inside the matrix branch of SimpleShampoo.step), after computing $u = su @ svT$, multiply u by $np.sqrt(u.shape[0] / u.shape[1])$ so that the operator's average output RMS for unit-RMS inputs matches the induced RMS-RMS norm used in your part-b metrics.^[18]



Can you give more specifics about the Shampoo update and its Frobenius Norm? Like what exactly is the Frobenius norm since the update matrix is orthogonal?

For the simplified Shampoo update in this homework, the weight update for a dense layer is constructed by orthogonalizing the momentum matrix m via an SVD $m = U\Sigma V^T$ and then setting the update to $u = UV^T$. This means the update matrix u is (approximately) orthogonal: its columns and rows are orthonormal, so it represents a pure rotation/reflection in the space of activations rather than stretching more along some directions than others.

For any matrix A , the Frobenius norm is

$$\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2},$$

i.e., the Euclidean length of the vector of all entries, and it is equal to the square root of the sum of the squares of all singular values. If u is orthogonal and square with size $n \times n$, all of its singular values are 1, so^{[19] [20]}

$$\|u\|_F^2 = \sum_{i=1}^n 1^2 = n \quad \Rightarrow \quad \|u\|_F = \sqrt{n}, \quad \text{Correct.}$$

which depends only on the dimension, not on which rotation/reflection it is. This explains what you see in the Shampoo plots: because the update is orthogonalized, the spectral norm (largest singular value) is fixed at 1 while the Frobenius norm reflects only the size of the layer (and any extra scaling you apply), giving a relatively flat spectral norm but Frobenius norms that still vary mainly with width rather than with the particular gradient directions.^{[21] [22] [23] [24]}



1. [q_mup_coding.ipynb](#)
2. [q_mup_coding.ipynb](#)
3. [q_mup_coding.ipynb](#)
4. [q_mup_coding.ipynb](#)
5. [q_mup_coding.ipynb](#)
6. [q_mup_coding.ipynb](#)
7. <https://cvl-umass.github.io/compsci682-fall-2023/notes/neural-networks-1/>
8. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12133272/>
9. <https://www.v7labs.com/blog/neural-networks-activation-functions>
10. <https://www.geeksforgeeks.org/machine-learning/activation-functions-neural-networks/>

11. [https://en.wikipedia.org/wiki/Neural_network_\(machine_learning\)](https://en.wikipedia.org/wiki/Neural_network_(machine_learning))
12. <https://www.datacamp.com/tutorial/introduction-to-activation-functions-in-neural-networks>
13. <https://www.codecademy.com/article/understanding-neural-networks-and-their-components>
14. <https://encord.com/blog/activation-functions-neural-networks/>
15. https://www.reddit.com/r/learnmachinelearning/comments/12j4hxa/understanding_activation_functions_a_mustknow_for/
16. Screenshot-2025-12-09-at-7.28.50-PM.jpg
17. Screenshot-2025-12-09-at-7.28.59-PM.jpg
18. Screenshot-2025-12-09-at-7.35.53-PM.jpg
19. <https://www.sciencedirect.com/topics/mathematics/frobenius-norm>
20. <https://www.rohan-paul.com/p/ml-interview-q-series-how-can-you-103>
21. <https://www.cse.psu.edu/~b58/cse456/lecture3.pdf>
22. https://en.wikipedia.org/wiki/Orthogonal_matrix
23. <https://blogs.sas.com/content/iml/2025/01/08/matrix-norms-spectra.html>
24. <https://www.cs.umb.edu/~dsim/slidesNM.pdf>
25. <https://dspace.mit.edu/bitstream/handle/1721.1/75282/18-335j-fall-2006/contents/lecture-notes/lec2.pdf>
26. <https://www.cis.upenn.edu/~cis5150/cis515-11-sl4.pdf>

A really cool feature of perplexity is that it actually lists all of its sources and cites them so that you can check where its information is coming from or follow up with it. Overall I think it did a pretty good job!